

Examen scris**Detalii:** 1 punct din oficiu, 18 probleme fiecare valorează 0.5 puncte, timp de lucru 2 ore.

- I. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```

1. #include<iostream>
2. using namespace std;
3. class Base {
4. public:
5. int f() const { cout << "B1\n"; return 1; }
6. int f(string) const {
7. cout << "B2\n"; return 1; } };
8. class Derived : public Base {
9. public: int f() const {
10. cout << "D1\n"; return 2; } };
11. int main() {
12. string s("hello");
13. Derived d;
14. int x = d.f();
15. d.f(s);
16. return 0; }

```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

- II. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-d sau adevărat/fals).

II.1 Care este afirmația greșită referitoare la clasele abstracte?

- a) o clasă abstractă poate fi folosită ca tip pentru pointeri sau referințe;
- b) o clasă abstractă poate moșteni dintr-o altă clasă abstractă;
- c) o clasă abstractă trebuie să conțină doar funcții virtuale pure;
- d) o clasă abstractă poate avea funcții virtuale pure definite (implementate) în afara clasei, chiar dacă rămân virtuale pure

II.2 Care afirmație este corectă în cazul moștenirii protected?

- a) membrii *protected* ai clasei de bază devin *private* în clasa derivată
- b) membrii *public* ai clasei de bază nu mai pot fi accesați nici din clasa derivată
- c) conversia implicită de la un obiect al clasei derivate la clasa de bază este permisă doar în interiorul clasei derivate și al claselor derivate din aceasta
- d) membrii *private* ai clasei de bază devin *protected* în clasa derivată.

II.3 În C++, este recomandat să se arunce excepții sub forma pointerilor către obiecte, pentru a evita copierea acestora.

Adevărat Fals

II.4 Operatorul & aplicat unei referințe returnează adresa referinței însăși.

Adevărat Fals

II.5 Eliberarea printr-un pointer la clasa de bază (delete pb;) a unui obiect alocat dinamic din clasa derivată apelează întotdeauna destructorul clasei derivate.

Adevărat Fals

- III. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```

1. #include <iostream>
2. using namespace std;
3. struct C { int x;
4. int get() const { return 1; }
5. int get() { return 2; }
6. };
7. int main() {
8. const C c;
9. std::cout << c.get();
10. }

```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

IV. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class Baza{ public: virtual ~Baza(){} };
4. class D1 : virtual public Baza{};
5. class D2 : public D1{};
6. class D3 : virtual public Baza{};
7. class D4 : public D1, public D3{};
8. int main() {
9.     D4 ob;
10.    Baza& re = ob;
11.    try{ throw re; }
12.    catch(D1& o){cout<<"D1\n";}
13.    catch(D2& o){cout<<"D2\n";}
14.    catch(D3& o){cout<<"D3\n";}
15.    catch(D4& o){cout<<"D4\n";}
16.    catch(Baza& o){cout<<"Baza\n";}
17. }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

V. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-d sau adevărat/fals).

V.1 Care dintre funcțiile enumerate mai jos nu aparține clasei `type_info` din C++?

- a) Operatorul `==`
- b) Operatorul `!=`
- c) Funcția `name()`
- d) Funcția `type()`
- e) nici un răspuns corect

V.2 Ce se întâmplă dacă o excepție este aruncată într-o funcție și nu este prinsă în acea funcție?

- a) Programul continuă execuția normal
- b) Programul intră într-o buclă infinită
- c) Excepția se propagă către funcția apelată
- d) Excepția se propagă către funcția apelantă

V.3 Clasele locale din funcții nu pot avea funcții membre automat inline.

Adevărat Fals

V.4 Expresia `typeid(*pb)`, unde `pb` este un pointer la o clasă de bază fără funcții virtuale, returnează informația de tip corespunzătoare tipului dinamic (real) al obiectului indicat.

Adevărat Fals

V.5 Fie `Baza*` `pb` un pointer către un obiect al unei clase polimorfe. Dacă conversia `dynamic_cast<Derivata*>(pb)` eșuează, se aruncă excepția `std::bad_cast`, la fel ca în cazul conversiei eșuate `dynamic_cast<Derivata&>(rb)` pe referințe.

Adevărat Fals

VI. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class Cls{ public: void afis(){cout<<"1 ";}
4. Cls operator+(Cls ob){cout<<"2 "; return Cls();}
5. Cls operator-(Cls ob){cout<<"3 "; return Cls();}};
6. class Cls2 : public Cls{
7. public: void afis(){cout<<"4 ";}
8. Cls2 operator+(Cls2 ob){cout<<"5 "; return Cls2();} };
9. int main() {
10.    Cls a;
11.    Cls2 d,e,f;
12.    (d + e).afis();
13.    (d - e).afis();
14.    (a + d).afis();
15.    (d + a).afis();
16. }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr lini modificată și modificarea)

VII. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class FormaGeometrica {
4. public:
5.     void deseneaza() {
6.         std::cout << "Am desenat forma\n"; }
7. };
8. class Patrulater : public FormaGeometrica {};
9. class Paralelogram : public FormaGeometrica {};
10. class Patrat : public Patrulater, public Paralelogram {};
11. int main() {
12.     Patrat obj;
13.     obj.deseneaza();
14. }
```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

VIII. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-e sau adevărat/fals).

VIII.1 Ce se înțelege prin "supraîncărcarea constructorilor" (Constructor Overloading)?

- a) Definirea unui constructor în interiorul altui constructor.
- b) Apelarea unui constructor de prea multe ori într-o buclă repetitivă, ducând la *stack overflow*.
- c) Existența mai multor constructori în aceeași clasă, care au același nume dar liste de parametri diferite (număr sau tipuri de argumente).
- d) Modificarea comportamentului unui constructor dintr-o clasă de bază în clasa derivată.
- e) Declararea unui constructor cu modificatori de acces diferiți în același timp (ex: public și private simultan).

VIII.2 Care dintre următorii operatori **NU** poate fi supraîncărcat în C++?

- a) Operatorul de atribuire (=) b) Operatorul de indexare ([])
- c) Operatorul de rezoluție de scop (::) d) Operatorul de adunare (+)
- e) Operatorul de egalitate (==) f) Operatorul de inserție în flux (<<)

VIII.3 Supraîncărcarea funcțiilor este un caz de polimorfism la execuție.

Adevărat Fals

VIII.4 O funcție membră statică poate declara un obiect local al clasei și îi poate apela metodele.

Adevărat Fals

VIII.5 O variabilă de instanță constantă se inițializează în corpul constructorului (intre acolade).

Adevărat Fals

IX. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class B {
4. public: virtual void f()= 0;
5. };
6.
7. void B::f() { std::cout<< "B::f()"; }
8. class D: public B {
9.     void f(int x) {
10.         std::cout<< "D:: f(x) = " << x; }
11. };
12.
13. int main() {
14.     B *pb = new D();
15.     pb->f();
16.     return 0;
17. }
```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

- X. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1.  #include<iostream>
2.  using namespace std;
3.  int main()
4.  {int a = 1;
5.    int b = ++a;
6.    const int *a_ptr = &a;
7.    int *const b_ptr = &b;
8.    *b_ptr += 4;
9.    *a_ptr += 5;
10.   std::cout << *b_ptr << std::endl;
11.   std::cout << *a_ptr << std::endl;
12.   return 0; }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

- XI. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-g sau adevărat/fals).

XI.1 Constructorii în C++

- a) Constructorii pot returna o valoare de orice tip primitiv, inclusiv void.
- b) Dacă o clasă definește un constructor de copiere, compilatorul nu mai generează automat constructorul implicit, chiar dacă nu este definit explicit.
- c) Constructorul cu un singur parametru creează o conversie implicită de la tipul parametrului la tipul clasei.
- d) Răspunsurile a și b sunt corecte
- e) Răspunsurile a și c sunt corecte
- f) Răspunsurile b și c sunt corecte
- g) Răspunsurile a, b și c sunt corecte

XI.2 Template-urile în C++

- a) Instanțierea implicită a unui template are loc la momentul compilării, generând câte o versiune separată a codului pentru fiecare unitate de traducere în care este utilizat, indiferent de tipul folosit (o singură versiune).
- b) template-urile de clase nu admit specializare explicită
- c) Specializarea explicită permite un comportament diferit al template-ului pentru un tip anume
- d) nici un răspuns corect

XI.3 într-o clasă care conține un membru de tip referință (int& ref;), acest membru poate fi inițializat atât în lista de inițializare a constructorului, cât și prin atribuire în corpul constructorului, deoarece ambele variante au loc înainte de finalizarea construcției obiectului.

Adevărat Fals

XI.4 Intr-o funcție membră nestatică se poate apela o funcție membră statică.

Adevărat Fals

XI.5 O constantă statică declarată în clasă (static const int n;) trebuie definită în afara acesteia.

Adevărat Fals

- XII. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1.  #include <iostream>
2.  using namespace std;
3.  void f(int a, int b = 5, int c) {
4.    std::cout << a + b + c;
5.  }
6.  int main() {
7.    f(1, 2, 3);
8.  }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

XIII. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include<iostream>
2. using namespace std;
3. class B { int b;
4. public: B(int p = 1) { b = p; } };
5. class D : public B { int* d;
6. public: D(int p) { d = new int; *d = p; }
7. D(const D& s) : B(s) { d = new int; *d = *(s.d); }
8. ~D() { delete d; }
9. void set(int p) { *d = p; } };
10.
11. int main() {
12. D o1(2), o2(3);
13. o1 = o2;
14. o2.set(4);
    return 0; }
```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

XIV. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-d sau adevărat/fals).

XIV.1 Care dintre următoarele afirmații motivează corect redefinirea unui operator ca funcție independentă (non-membră) în loc de funcție membră a clasei?

- a) Funcțiile independente au acces direct la membrii privați ai clasei fără a fi necesară declararea lor ca friend, spre deosebire de funcțiile membre.
- b) Redefinirea ca funcție independentă permite ca operandul stâng să fie de un tip diferit de cel al clasei, făcând posibile expresii de forma $5 + ob$, unde primul operand este un tip primitiv.
- c) Operatorii definiți ca funcții independente sunt apelați mai eficient de către compilator: nu implică pasarea implicită a pointerului `this`.
- d) Standardul C++ impune că operatorii binari simetrici, precum `+` sau `*`, trebuie obligatoriu definiți ca funcții independente pentru a respecta comutativitatea.

XIV.2 În care dintre următoarele situații se folosește cuvântul `virtual` în C++?

- a) la declararea funcțiilor virtuale pentru activarea RTTI
- b) la declararea variabilelor membre ale unei clase pentru a întârzia alocarea lor în memorie până la prima utilizare efectivă
- c) la moștenirea virtuală pentru rezolvarea ambiguității cauzate de moștenirea multiplă
- d) răspunsurile a și b sunt corecte
- e) răspunsurile a și c sunt corecte
- f) răspunsurile a, b, și c sunt corecte

XIV.3 Într-o funcție membră statică se pot folosi date membre nestatice

Adevărat Fals

XIV.4 Struct nu poate avea funcții membre virtuale.

Adevărat Fals

XIV.5 O funcție statică nu poate să fie virtuală.

Adevărat Fals

XV. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class B {public:
4. virtual B* fv() { return this; }
5. int adun(int p) { return p + 1; } };
6. class D : public B {public:
7. virtual D* fv() { return this; }
8. int adun(int p) { return p + 2; } };
9. int main() {
10. B* p = new D;
11. int x = p->fv()->adun(1);
12. std::cout << x << "\n";
13. return 0; }
```

Programul compilează? DA ☐ NU ☐

Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

XVI. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include<iostream>
2. using namespace std;
3. class C {int c;
4. public: C(int p = 1) { c = p; }
5.         int& get() const { return c; } };
6.
7. int f(C op) { return op.get(); }
8.
9. int main() {
10.  C ol;
11.  int x = f(ol);
12.  std::cout << x << "\n";
13.  return 0; }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)

XVII. Exercițiu de teorie, problemele grila au un singur răspuns corect/cel mai potrivit. Fiecare exercițiu valorează 0.1 puncte, încercuiți răspunsul corect (varianta a-d sau adevărat/fals).

XVII.1 Obiecte statice în funcții

- a) Obiectele statice din funcții se redefinesc în afara funcției
- b) Obiectele statice din funcții apelează constructorul înaintea apelului funcției (similar cu obiectele globale)
- c) Obiectele statice din funcții apelează constructorul la prima intrare în funcție iar destructorul la finalul programului
- d) Obiectele statice din funcții apelează constructorul și destructorul la fiecare apel al funcției
- e) Nici un răspuns corect

XVII.2 Excepțiile în C++

- a) Cuvintele cheie folosite sunt catch, exception, try
- b) Se pot „prinde” excepții și pe tipuri de date încă nedefinite prin catch (...)
- c) Întâi se pune tipul de baza și apoi tipul derivat în lista de catch-uri
- d) Nici un răspuns corect

XVII.3 Clasele „nested”(clasa în clasa) pot avea date membre statice.

Adevărat Fals

XVII.4 Clasele „nested”(clasa în clasa) nu pot avea funcții membre statice

Adevărat Fals

XVII.5 Clasele locale din funcții pot accesa variabilele locale ale acestora

Adevărat Fals

XVIII. Spuneți dacă programul de mai jos este corect. În caz afirmativ, spuneți ce afișează, în caz negativ propuneți o (singură) modificare prin care programul devine corect.

```
1. #include <iostream>
2. using namespace std;
3. class C { int a;
4.         static int x;
5. public:
6.         C(int a = 22) { x++; this->a=a; }
7.         static int f() {return x;}
8.         int getA(){return a;}
9. }a;
10. int C::x=20;
11. int main() {
12.  C a(33),b;
13.  cout<<"instantieri C: "<<C::f()<<" val elem:"<<::a.getA();
14.  return 0; }
```

Programul compilează? DA ☐ NU ☐
Dacă DA ce se afișează pe ecran:

Dacă NU: de ce nu?

modificarea care îl face să meargă (o singură linie modificată, precizat nr linie modificată și modificarea)